

Relatório de Auditoria de Segurança

Projeto: FutStats (futebol-analise) | Data: 03 de Setembro de 2026

Escopo Auditado: Repositório completo (frontend Vanilla JS PWA, Vercel Serverless Functions em Node.js, Deno Edge Function e scripts de banco Supabase PostgreSQL).

Nota Metodológica e Mapeamento para a Stack:

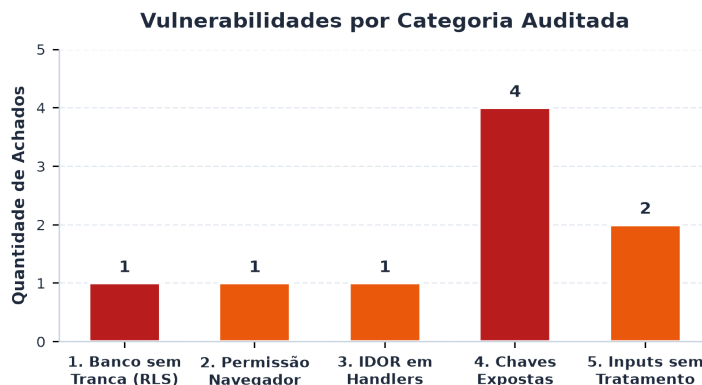
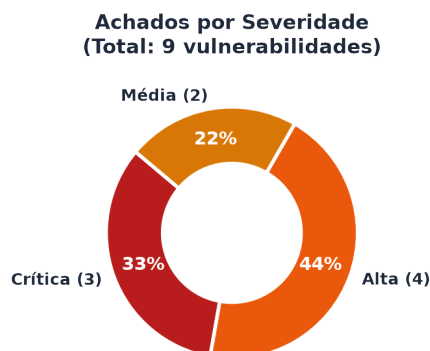
A auditoria inspecionou rigorosamente o código-fonte em busca de 5 categorias essenciais de segurança, adaptadas à arquitetura serverless/client-side do FutStats:

- 1. BANCO SEM TRANCA (Isolamento de Inquilino/Dono):** Avaliação das políticas de Row Level Security (RLS) no Supabase PostgreSQL (tabela push_subscriptions) e análise de queries/operações anônimas de listagem, inserção e deleção.
- 2. PERMISSÃO DEFINIDA NO NAVEGADOR:** Confronto entre o controle de rotas no cliente e a validação de privilégios de execução no backend para rotas críticas (como o disparador em massa do robô de alertas em api/cron-alerts.js).
- 3. IDOR (Insecure Direct Object Reference):** Verificação de todos os handlers de rota em api/ para identificar busca, alteração ou exclusão de objetos baseados em parâmetros sem comprovação de posse ou autenticação.
- 4. CHAVES EXPOSTAS (Hardcode & Defaults):** Varredura estática de API keys, tokens JWT, chaves privadas VAPID e credenciais de deploy embutidas em código, fallbacks, bundles públicos e histórico git.
- 5. INPUTS SEM TRATAMENTO (XSS):** Inspeção de todas as 94 atribuições a innerHTML no frontend, atributos href/src com dados externos e validação de esquemas de protocolo (ex: javascript:).

Status da Avaliação: AÇÃO REQUERIDA	Total de Achados: 9
Vulnerabilidades Críticas: 3	Vulnerabilidades Altas: 4
Vulnerabilidades Médias: 2	Pontos Fortes Validados: 6

1. Resumo Executivo

A auditoria de segurança identificou **9 vulnerabilidades confirmadas** no código-fonte, das quais **3 são Críticas**, **4 são Altas** e **2 são Médias**. Os riscos centrais decorrem de políticas permissivas de RLS no banco Supabase que expõem assinaturas a extração ou deleção em massa, chaves privadas (VAPID) e de API gravadas no código como valores de fallback, ausência de restrição de acesso por padrão no robô de alertas (Fail-Open), manipulação de inscrições por IDOR e ausência de sanitização de protocolo em links de notícias externas (XSS via pseudo-protocolo).



2. Pontos Fortes e Riscos Centrais

[+] Pontos Fortes Verificados no Código:

- **Proteção SSRF no Proxy de Imagens (api/img.js):** O endpoint valida rigorosamente se a URL solicitada inicia com `https://media.api-sports.io/`, impedindo acesso a redes internas (ex: metadados de nuvem 169.254.169.254).
- **Lista Branca de Endpoints no Proxy da API (api/football.js):** Implementação de um Set estrito (`ALLOWED_ENDPOINTS`) que rejeita qualquer tentativa de consulta a endpoints administrativos ou não autorizados da API-Sports.
- **Escape HTML Generalizado no Frontend (public/app.js):** A função `escapeHtml()` é empregada em dezenas de pontos de exibição dinâmica (nomes de jogadores, times, placares e rodadas), prevenindo XSS tradicional em tags HTML.
- **Ausência de Avaliação Dinâmica (Eval):** Nenhuma chamada a `eval()`, `new Function()` ou scripts dinâmicos de terceiros foi detectada no código de produção.
- **Respostas Estruturadas JSON no Backend:** Todas as Serverless Functions respondem com `res.json()` ou buffers de imagem, eliminando injeção de HTML no lado do servidor.
- **Tratamento Seguro de Push no Service Worker (public/sw.js):** O evento push realiza parsing seguro de JSON sem executar scripts embutidos no corpo da notificação.

[-] Pontos Fracos (Riscos Centrais):

- **Vazamento e Deleção Global no Supabase (RLS Aberta):** Políticas com `FOR SELECT TO anon USING (true)` e `FOR DELETE TO anon USING (true)` tornam o banco de inscrições totalmente acessível e destrutível por qualquer usuário.
- **Comprometimento Criptográfico WebPush:** A presença da chave privada VAPID no código permite falsificação e envio de notificações push maliciosas a todos os dispositivos cadastrados.
- **Execução Não Autorizada do Robô de Alertas (Fail-Open):** Ausência de validação obrigatória de `CRON_SECRET` permite que terceiros sobrecarreguem a cota da API-Sports e façam spam push.
- **Ausência de Validação de Protocolo de URL (XSS):** Links provenientes de feeds RSS são inseridos em tags `<a href="...">` sem checagem de esquema `http/https`.

3. Tabela Detalhada de Achados por Categoria

Abaixo estão listadas todas as 9 vulnerabilidades verificadas diretamente no código-fonte:

Sev.	Categoria	Arquivo e Linha(s)	Descrição e Impacto
CRÍTICA	1. Banco sem Tranca	supabase/schema_rls.sql:33-47	Políticas RLS permitem que qualquer cliente anônimo faça SELECT irrestrito (extraindo todos os tokens push e aparelhos) e DELETE global (limpando a base de dados).
CRÍTICA	4. Chaves Expostas	api/cron-alerts.js:14 api/subscribe.js:11	Chave Privada VAPID embutida como fallback no código-fonte. Permite assinatura e envio forjado de pushes maliciosos a todos os usuários da aplicação.
CRÍTICA	4. Chaves Expostas	api/cron-alerts.js:8 .env.local:2	Chave de Produção da API-Sports hardcoded no código e no arquivo de ambiente local. Risco de exaustão de cota contratada e custos indevidos.
ALTA	2. Permissão no Navegador	api/cron-alerts.js:30-37	Fail-Open na verificação de CRON_SECRET : se a variável não estiver definida, a checagem é ignorada. Permite acionamento público ilimitado do robô de alertas.
ALTA	3. IDOR	api/subscribe.js:30-47, 60-70	Handler de deleção e atualização aceita qualquer endpoint sem autenticação ou validação de posse, permitindo exclusão ou adulteração de inscrições alheias.
ALTA	4. Chaves Expostas	.env.local:3	Token de Autenticação Vercel OIDC gravado em arquivo local commitável, contendo identificadores da equipe, usuário e projeto de deploy.
ALTA	5. Inputs sem Tratamento	public/app.js:1438	XSS via URL Scheme em links de notícias : <code>escapeHtml(item.link)</code> não valida se o protocolo inicia com <code>http/https</code> , permitindo <code>javascript</code> .
MÉDIA	5. Inputs sem Tratamento	public/app.js:6077, 6140	Interpolação de URLs de escudos e logos lidos de <code>localStorage</code> em atributos <code>src</code> de tags <code></code> sem sanitização ou <code>escape</code> .
MÉDIA	4. Chaves Expostas	Histórico Git (commits 1c6bb3c, 2047590)	Credenciais e tokens presentes no histórico de alterações do Git. Mesmo removidos da árvore atual, permanecem acessíveis na árvore histórica.

4. Recomendações Priorizadas de Correção

Prioridade 1 (P1 — Imediato / Crítico):

1. **Ajustar Políticas RLS no Supabase:** Revogar SELECT e DELETE irrestritos para a role anon. O robô de alertas deve consultar a tabela utilizando estritamente a service_role (com chave segura no ambiente da Vercel).
2. **Rotacionar Chaves e Eliminar Fallbacks Hardcoded:** Gerar um novo par de chaves VAPID (web-push generate-vapid-keys) e revogar a chave da API-Sports. Cadastrar as novas chaves exclusivamente nas variáveis de ambiente da Vercel (sem defaults no código).

Prioridade 2 (P2 — Curto Prazo / Alto):

3. **Impor Fail-Closed no Robô de Alertas (api/cron-alerts.js):** Exigir obrigatoriamente a presença de CRON_SECRET ou validação do header x-vercel-cron para autorizar a execução, retornando HTTP 401 se ausente.
4. **Sanitização de URLs Externas (public/app.js):** Implementar a função sanitizeUrl() validando que qualquer link externo de notícias inicie estritamente com http:// ou https:// antes de renderizar em tags <a href>.
5. **Proteção IDOR em Subscrições:** Exigir validação criptográfica (ex: conferência de auth secret ou hash da inscrição) antes de processar exclusões ou atualizações em api/subscribe.js.

Prioridade 3 (P3 — Médio Prazo / Higiene de Segurança):

6. **Expurgo do Histórico Git:** Executar git-filter-repo ou BFG Repo-Cleaner para remover commits contendo chaves antigas e forçar novo push com histórico limpo.
7. **Remoção do Token OIDC:** Excluir VERCEL_OIDC_TOKEN de .env.local e incluir o arquivo no .gitignore caso ainda não esteja.

5. Issues Prontas para o GitHub

Abaixo estão formatadas as issues prontas para abertura no repositório GitHub. Cada bloco contém o texto completo em Markdown delimitado.

--- ISSUE 1 ---

Título: [Segurança] [Crítica] Políticas RLS permissivas expõem tabela push_subscriptions a vazamento e deleção em massa

Labels sugeridas: security, severidade:critica, database, rls

Descrição do Problema: A tabela push_subscriptions no Supabase possui políticas de RLS excessivamente permissivas configuradas para a role 'anon'. Qualquer requisitante não autenticado pode executar um SELECT irrestrito para baixar todas as inscrições push (incluindo endpoints, chaves criptográficas p256dh e auth) ou executar um DELETE sem restrições, apagando todas as inscrições da base.

Evidência de Código:

Arquivo: supabase/schema_rls.sql:33-47

```
CREATE POLICY "Permitir delete anonimo por endpoint" ON push_subscriptions FOR DELETE TO anon USING (true);  
CREATE POLICY "Permitir select de assinaturas" ON push_subscriptions FOR SELECT TO anon, service_role USING (true);
```

Impacto: Exposição de dados privados de navegação e aparelhos de todos os usuários, além de Negação de Serviço (DoS) permanente com apagamento de todos os assinantes.

Sugestão de Correção:

1. Remover permissão de SELECT da role 'anon'. Apenas a 'service_role' deve ter acesso a SELECT.
2. Restringir DELETE para exigir correspondência com o endpoint informado no cabeçalho ou body validado.
3. Configurar SUPABASE_SERVICE_ROLE_KEY no painel da Vercel para que o robô api/cron-alerts consulte os assinantes de forma segura.

Critérios de Aceite:

- [] SELECT na tabela push_subscriptions retorna vazio/negado para a role anon.
- [] Operações de DELETE irrestritas sem filtro são bloqueadas pelo banco.
- [] O robô de alertas na Vercel utiliza service_role autenticada para leitura.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] [Crítica] Chave Privada VAPID e chave da API-Football hardcoded no código-fonte

Labels sugeridas: security, severidade:critica, secrets, backend

Descrição do Problema: As variáveis FOOTBALL_API_KEY e VAPID_PRIVATE_KEY possuem strings de credenciais reais embutidas como valores de fallback direto nos arquivos de backend (api/cron-alerts.js e api/subscribe.js), além de estarem gravadas em .env.local. A posse da VAPID_PRIVATE_KEY permite a qualquer terceiro gerar e assinar notificações WebPush fraudulentas em nome do FutStats para os aparelhos inscritos.

Evidência de Código:

```
Arquivo: api/cron-alerts.js:8, 14 | api/subscribe.js:11
const FOOTBALL_API_KEY = process.env.FOOTBALL_API_KEY || "a70fc65a67c10981ace9813a509db554";
const VAPID_PRIVATE_KEY = process.env.VAPID_PRIVATE_KEY ||
"gwKwUrc5XBpYU0BExM1ha_M3ugoo5JbM7mQSMt4Lk_c";
```

Impacto: Falsificação de notificações WebPush oficiais (phishing, malware), exaustão de cota na API-Sports e perda de controle da identidade criptográfica da aplicação.

Sugestão de Correção:

1. Rotacionar imediatamente as chaves VAPID (npx web-push generate-vapid-keys).
2. Revogar e gerar nova chave na API-Sports.
3. Remover todos os fallbacks do código-fonte e exigir validação de inicialização que encerre o processo caso as variáveis não estejam definidas na Vercel.
4. Adicionar .env* ao .gitignore.

Critérios de Aceite:

- [] Nenhuma chave ou token em texto plano nos arquivos .js.
- [] Aplicação encerra com erro amigável se variáveis obrigatórias não existirem.
- [] Novas chaves cadastradas estritamente nas variáveis de ambiente da Vercel.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] [Alta] Fail-Open no robô de alertas permite execução não autorizada e exaustão de recursos

Labels sugeridas: security, severidade:alta, authorization, backend

Descrição do Problema: A verificação de autorização em `api/cron-alerts.js` está encapsulada dentro de um bloco `'if (process.env.CRON_SECRET)'`. Se a variável não estiver configurada no ambiente, o bloco é pulado e a rota aceita qualquer requisição HTTP externa sem autenticação.

Evidência de Código:

```
Arquivo: api/cron-alerts.js:30-37
if (process.env.CRON_SECRET) {
  const authHeader = req.headers["authorization"] || "";
  const isVercelCron = req.headers["x-vercel-cron"] === "1";
  if (!isVercelCron && authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return res.status(401).json({ error: "Acesso não autorizado..." });
  }
}
```

Impacto: Exaustão da cota da API externa via chamadas repetitivas, disparo indevido de pushes repetidos (spam) e custos financeiros adicionais.

Sugestão de Correção:

1. Implementar padrão Fail-Closed: a rota deve SEMPRE exigir autorização válida.
2. Rejeitar com 401 se a requisição não possuir o cabeçalho `x-vercel-cron` da Vercel ou o Bearer token configurado.

Critérios de Aceite:

- [] Requisições externas diretas sem token retornam 401 Unauthorized.
- [] Apenas Vercel Cron ou clientes autorizados conseguem disparar o ciclo.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] [Alta] IDOR na gestão de inscrições em `api/subscribe.js`

Labels sugeridas: security, severidade:alta, idor, backend

Descrição do Problema: O endpoint `api/subscribe.js` aceita requisições DELETE e POST informando apenas a URL do 'endpoint' push, sem autenticar o requisitante nem verificar a posse da chave 'auth'. Isso permite que um atacante que conheça o endpoint de outro torcedor delete ou altere arbitrariamente sua inscrição.

Evidência de Código:

```
Arquivo: api/subscribe.js:30-47
if (req.method === 'DELETE') {
  const endpoint = req.query.endpoint || (req.body && req.body.endpoint);
  await fetch(`${SUPABASE_URL}/rest/v1/push_subscriptions?endpoint=eq.${encodeURIComponent(endpoint)}`,
    ...);
}
```

Impacto: Manipulação e exclusão não autorizada de inscrições de outros usuários (IDOR).

Sugestão de Correção:

1. Exigir que a requisição de DELETE inclua a chave criptográfica `auth` correspondente para validar a titularidade do aparelho.
2. No POST de atualização, validar se o endpoint já existente confere com a chave `auth` antes de sobrescrever preferências.

Critérios de Aceite:

- [] Deleção de endpoint exige correspondência com a chave `auth`.
- [] Não é possível sobrescrever preferências de endpoints de terceiros sem a chave criptográfica.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---**Título:** [Segurança] [Alta] XSS via esquema de URL javascript: em links de matérias externas**Labels sugeridas:** security, severidade:alta, frontend, xss

Descrição do Problema: A função `escapeHtml()` em `public/app.js` substitui apenas caracteres HTML especiais (`<`, `>`, `"`, `'`, `&`) mas não valida o esquema do protocolo da URL. Na renderização de notícias esportivas recebidas de feeds RSS, o link do item é atribuído diretamente ao atributo `href` sem validar se inicia com `http://` ou `https://`, viabilizando execução de código via javascript:.

Evidência de Código:Arquivo: `public/app.js:1438`

```
<a class="news-card-item" href="{escapeHtml(item.link)}" target="_blank" rel="noopener noreferrer" ...>
```

Impacto: Execução arbitrária de scripts no navegador da vítima caso um feed RSS externo ou payload forjado contenha um link com protocolo javascript:.

Sugestão de Correção:

1. Criar a função `sanitizeUrl(url)` que valida estritamente a URL contra protocolos permitidos (`http:` e `https:`).
2. Aplicar `sanitizeUrl()` em todos os atributos `href` que recebem dados dinâmicos.

Critérios de Aceite:

[] Links com protocolo javascript: são convertidos para '#' ou bloqueados.

[] Apenas URLs seguras (`http/https`) são renderizadas no atributo `href`.

--- FIM ISSUE 5 ---**--- ISSUE 6 ---****Título:** [Segurança] [Média] Token OIDC da Vercel exposto em arquivo de configuração local**Labels sugeridas:** security, severidade:media, git, devops

Descrição do Problema: O arquivo `.env.local` contém uma variável `VERCEL_OIDC_TOKEN` com um token JWT assinado pela Vercel contendo identificadores da conta de equipe, usuário e projeto de deploy.

Evidência de Código:Arquivo: `.env.local:3`

```
VERCEL_OIDC_TOKEN="eyJhbGciOiJSUzI1NiIs..."
```

Impacto: Exposição de metadados internos de infraestrutura da equipe de desenvolvimento na Vercel.

Sugestão de Correção:

1. Remover `VERCEL_OIDC_TOKEN` do arquivo `.env.local`.
2. Assegurar que `.env.local` e arquivos de ambiente estejam listados no `.gitignore`.

Critérios de Aceite:[] Arquivo `.env.local` não contém tokens sensíveis commitados.[] `.gitignore` inclui todos os padrões de arquivos de ambiente.

--- FIM ISSUE 6 ---